

MANUAL TÉCNICO

DE INFRAESTRUCTURA

Sistema PinkyPuff

Gestión de pedidos de ropa y accesorios

Guía de instalación, configuración y puesta en marcha

Universidad de las Fuerzas Armadas ESPE
Departamento de Ciencias de la Computación
Julio de 2026

Tabla de contenidos

Tabla de contenidos	2
Propósito del documento	4
1. Qué se va a levantar	4
Pila tecnológica.....	4
2. Requisitos previos	5
3. Ruta rápida	5
4. Paso 1 — Obtener el proyecto.....	5
Estructura relevante para la infraestructura	6
5. Paso 2 — Levantar PostgreSQL con Docker	6
5.1 Crear el docker-compose.yml.....	6
5.2 Levantar el contenedor.....	7
5.3 Comprobar que responde.....	7
6. Paso 3 — Instalar el esquema de la base de datos.....	7
Qué instala el script.....	7
Dos requisitos que impone el script.....	8
Verificar la instalación.....	8
7. Paso 4 — Configurar la conexión.....	8
Tres formas de crearlo	8
Por qué está fuera del JAR.....	9
Qué configura entonces persistence.xml.....	9
8. Paso 5 — Compilar con Maven.....	9
Qué hace cada plugin	9
Perfiles de JavaFX.....	10
9. Paso 6 — Crear el primer administrador	10
Opción de desarrollo	10
Opción de producción	10
10. Paso 7 — Ejecutar y verificar	11
Qué debe ocurrir, en orden	11
11. Verificación de la infraestructura	11
Consultas útiles de verificación	12
Alternativa gráfica: pgAdmin.....	12
12. Empaquetado y entrega al cliente	12
13. Operación y mantenimiento.....	13
Ciclo de vida del contenedor	13
Mantenimiento de la base.....	13
Actualizar el esquema.....	13
Cambiar de servidor de base de datos sin recompilar	13

Desbloquear una cuenta.....	13
14. Resolución de problemas.....	13
Depuración de consultas.....	14
15. Referencia rápida de comandos.....	14
Base de datos	14
Compilar y ejecutar	15
Mantenimiento de la base.....	15
Rutas clave de infraestructura.....	15
16. Advertencias sobre archivos engañosos.....	15

Para generar el índice en Word: clic derecho sobre esta tabla → «Actualizar campos» → «Actualizar toda la tabla».

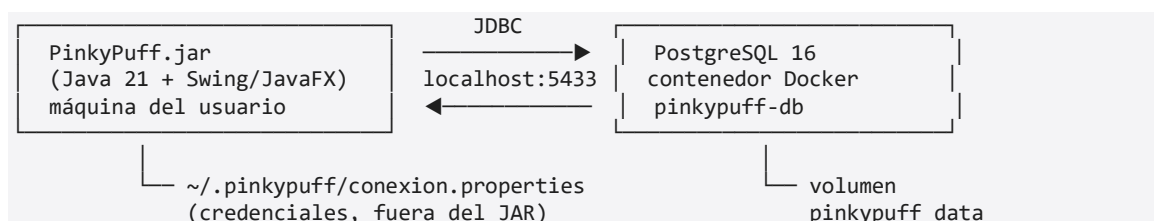
Propósito del documento

Este manual permite dejar el sistema PinkyPuff funcionando desde cero, en una máquina limpia y sin conocimiento previo del código. Cubre requisitos, base de datos, configuración de conexión, compilación, ejecución, verificación, empaquetado para el cliente y resolución de problemas.

Para entender **cómo está hecho** el código, consulte los documentos [MANUAL_TECNICO.md](#) y [DOCUMENTACION_TECNICA.md](#) del proyecto. Este manual responde a **cómo se levanta**.

1. Qué se va a levantar

PinkyPuff es una **aplicación monolítica de escritorio**. No hay servidor de aplicaciones, ni API HTTP, ni despliegue en la nube. La infraestructura se reduce a dos piezas: el ejecutable Java y un contenedor de PostgreSQL.



Dos hechos condicionan todo el proceso de instalación:

- **La base de datos es "gruesa"**. Buena parte de la lógica vive en PostgreSQL: funciones ([fn_autenticar_admin](#), [fn_evolucion_mensual](#), [fn_buscar_clientes](#)), procedimientos ([sp_refrescar_dashboard](#), [sp_mantenimiento_bd](#)), triggers de auditoría y consistencia, y vistas materializadas para el panel de inicio. Por eso no basta con que Hibernate cree las tablas: hay que ejecutar [dist/instalar_bd.sql](#) sí o sí.
- **Docker se usa solo en desarrollo**, para tener PostgreSQL sin instalarlo en la máquina. La aplicación no se contiene ni se despliega en Docker.

Pila tecnológica

Capa	Tecnología
Presentación	Java Swing (NetBeans GUI Builder) + FlatLaf + JavaFX
Controladores de UI	Clases propias en controller/
Negocio	service/ + funciones almacenadas de PostgreSQL
Persistencia	JPA 3.2 / Hibernate ORM 7
Datos	PostgreSQL 14+ (probado en 16, ejecutado en Docker)
Reportes	OpenPDF
Build	Maven, Java 21

2. Requisitos previos

Requisito	Versión	Por qué	Verificación
JDK	21 (LTS)	pom.xml fija maven.compiler.release = 21; el código usa switch con expresión	java -version
Maven	3.8+	Descarga dependencias, compila y empaqueta	mvn -version
Docker	Desktop o Engine + Compose	Ejecuta PostgreSQL 16	docker compose version
Git	cualquiera	Clonar el repositorio	git --version
NetBeans	opcional	Solo si se editan los formularios .form	—

- **El JDK debe ser 21.** No 17, no 22. El `.exe` generado para Windows incluye una comprobación: si no encuentra Java 21, abre la página de descarga de Adoptium.
- **Conexión a internet en la primera compilación.** Maven descarga Hibernate, el driver de PostgreSQL, Lombok, OpenPDF, FlatLaf y JavaFX. A partir de ahí queda todo en el repositorio local (`~/ .m2`).
- **No hace falta instalar PostgreSQL** en la máquina si se usa Docker. Si ya hay uno instalado no habrá conflicto: el contenedor se expone en el puerto **5433**, precisamente para evitar chocar con el 5432 nativo.

3. Ruta rápida

Resumen ejecutable para quien ya conoce el terreno. Cada paso está explicado en detalle en las secciones siguientes.

```
# 1. Proyecto
git clone <URL_DEL_REPOSITORIO>
cd mavenproject1

# 2. Crear docker-compose.yml (ver sección 5 – no está versionado)

# 3. Base de datos
docker compose up -d
docker ps
docker exec -i pinkypuff-db psql -U admin -d postgres < dist/instalar_bd.sql

# 4. Compilar
mvn clean package                # → target/PinkyPuff.jar

# 5. Ejecutar (el primer arranque pide las credenciales en un diálogo)
java -jar target/PinkyPuff.jar
```

4. Paso 1 — Obtener el proyecto

Clone el repositorio y sitúese en la raíz del módulo Maven (`mavenproject1/`, la carpeta que contiene `pom.xml`). Todos los comandos de este manual se ejecutan desde ahí.

Estructura relevante para la infraestructura

mavenproject1/	
├ pom.xml	Build, dependencias, perfiles de distribución
├ docker-compose.yml	← HAY QUE CREARLO (no está versionado)
└ schema.sql	Esquema mínimo, histórico. NO instalar con esto.
├ src/main/resources/	
│ └ META-INF/persistence.xml	Unidad de persistencia REAL (sin credenciales)
├ target/	Salida de Maven (PinkyPuff.jar, PinkyPuff.exe)
└ dist/	Paquete listo para entregar al cliente
├ PinkyPuff.jar / .exe	
├ PinkyPuff.sh / .bat	Lanzadores
├ instalar_bd.sql	← Script AUTORITATIVO de la base de datos
├ conexion.properties.ejemplo	
└ INSTALACION.md	Manual del instalador

ANTES DEL PRIMER COMMIT

El repositorio no tiene historial todavía y en la raíz conviven varios archivos .zip pesados (uno de 105 MB) y un volcado de base de datos. Antes del primer commit conviene añadir un .gitignore con target/, dist/*.jar, dist/*.exe y *.zip. Si esos archivos entran al historial de Git, ya no salen.

5. Paso 2 — Levantar PostgreSQL con Docker

5.1 Crear el docker-compose.yml

El archivo **no está versionado en el repositorio**; cada quien lo crea en su copia local a partir del contenido del [README.md](#). Si se trabaja en equipo, conviene versionarlo para que todos partan de lo mismo.

Cree `docker-compose.yml` en la raíz del proyecto:

```
services:
  postgres:
    image: postgres:16
    container_name: pinkypuff-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: contra
      POSTGRES_DB: postgres
    ports:
      - "5433:5432"          # 5433 en el host para no chocar con PostgreSQL local
    volumes:
      - pinkypuff_data:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  pinkypuff_data:
```

Tres decisiones que conviene no cambiar a la ligera:

- **El puerto 5433 es deliberado.** Dentro del contenedor PostgreSQL escucha en 5432; el mapeo `5433:5432` lo expone en 5433 del host. Ese 5433 es el valor por defecto en [ConfiguracionBD](#) y en [conexion.properties.ejemplo](#). Si lo cambia, tendrá que cambiarlo también en `~/pinkypuff/conexion.properties`.
- `container_name: pinkypuff-db`. Todos los comandos `docker exec` de este manual usan ese nombre.
- **Volumen `pinkypuff_data`.** Es lo que hace que los datos sobrevivan a `docker compose down`.

5.2 Levantar el contenedor

```
docker compose up -d      # levantar en segundo plano
docker ps                 # comprobar que pinkypuff-db aparece como "Up"
```

5.3 Comprobar que responde

```
pg_isready -h localhost -p 5433
```

O directamente abriendo una consola SQL:

```
docker exec -it pinkypuff-db psql -U admin -d postgres
```

Debería aparecer el prompt `postgres=#`. Salga con `\q`. En Windows, si prefiere entrar primero al contenedor:

```
docker exec -it pinkypuff-db bash
psql -U admin -d postgres
```

6. Paso 3 — Instalar el esquema de la base de datos

PASO CRÍTICO

Este es el paso que más se olvida y el que más problemas causa. Hibernate crea las tablas, pero NO las funciones ni los triggers. Sin este script la aplicación arranca y muere en el login.

Con el contenedor levantado, desde la carpeta del proyecto:

```
docker exec -i pinkypuff-db psql -U admin -d postgres < dist/instalar_bd.sql
```

En PowerShell, si la redirección `<` da problemas:

```
Get-Content dist\instalar_bd.sql | docker exec -i pinkypuff-db psql -U admin -d postgres
```

Qué instala el script

`dist/instalar_bd.sql` (≈1470 líneas) es el script de referencia y es **idempotente**: se puede volver a ejecutar sobre una base ya creada para actualizarla.

Sección	Contenido
1	Esquema base: 7 tablas (administrador, clientes, pedidos, productos, etiquetas_config, producto_atributos, producto_etiquetas)
2.1	Extensiones: pg_trgm (búsqueda por similitud), pgcrypto, pg_stat_statements
3	Tablas de apoyo: sesiones_admin (bitácora de login), pedido_resumen (totales precalculados), auditoria_pedidos, dashboard_estado
4	27 índices, varios parciales (WHERE estado = 'PENDIENTE') y uno GIN de trigramas sobre clientes.nombre
5	Vistas: v_pedidos_detalle, v_productos_completo, v_sesiones_activas, v_inventario_libre
6	Vistas materializadas mv_dashboard_inicio y mv_top_productos_mes + sp_refrescar_dashboard
7	Funciones de reporte: fn_evolucion_mensual, fn_top_etiquetas, fn_ranking_clientes, fn_historial_cliente

Sección	Contenido
8	Triggers: fechas automáticas, coherencia de estado, recálculo de pedido_resumen, auditoría, marca de "sucio", normalización de nombres de cliente
9	Operaciones: fn_autenticar_admin, fn_dashboard_inicial, sp_cobrar_pedido, fn_agregar_producto, fn_buscar_clientes, sp_cerrar_sesion
10	Mantenimiento: sp_purgar_bitacoras, sp_mantenimiento_bd

Dos requisitos que impone el script

1. **`CREATE EXTENSION` normalmente exige superusuario.** Con el docker-compose.yml de arriba, el usuario `admin` es superusuario del contenedor y no hay problema. Si en cambio apunta a un PostgreSQL corporativo donde el usuario de la aplicación no es superusuario, un administrador deberá crear antes `pg_trgm` y `pgcrypto`.
2. **Si el script no se ejecuta, la aplicación arranca pero falla con function**
`fn_autenticar_admin(...) does not exist` o `function fn_evolucion_mensual(...) does not exist` en cuanto se abre el login o el reporte.

Verificar la instalación

```
docker exec -it pinkypuff-db psql -U admin -d postgres

\dt          -- deben aparecer las 7 tablas base + las de apoyo
\df fn_*     -- deben aparecer las funciones
\dm          -- vistas materializadas
\dx         -- extensiones
\q
```

ARCHIVO EQUIVOCADO

No use schema.sql de la raíz para instalar. Es una versión reducida y anterior: sirve para entender las tablas de un vistazo, no para crear la base.

7. Paso 4 — Configurar la conexión

Las credenciales **no están dentro del JAR ni en `persistence.xml`**. Viven en un archivo externo:

```
~/pinkypuff/conexion.properties      (Linux / macOS)
C:\Users\USUARIO\pinkypuff\conexion.properties  (Windows)
```

Contenido, con los valores por defecto coherentes con el `docker-compose.yml`:

```
db.host=localhost
db.puerto=5433
db.base=postgres
db.usuario=admin
db.password=contra
```

Tres formas de crearlo

3. **Dejar que la aplicación lo cree (recomendada).** En el primer arranque, si el archivo no existe, `DialogoConexion` asume primer arranque y abre un diálogo modal pidiendo host, puerto, base,

usuario y clave, con un botón "Probar conexión" que valida en segundo plano. Al guardar escribe el archivo y reinicia la fábrica de Hibernate. No hay que preparar nada por adelantado.

4. **Copiar la plantilla:** `dist/conexion.properties.ejemplo` → `~/.pinkypuff/conexion.properties`.
5. **Apuntar a otra ruta al arrancar:** `java -Dpinkypuff.config=/ruta/a/conexion.properties -jar PinkyPuff.jar`

Por qué está fuera del JAR

Antes las credenciales vivían dentro de `persistence.xml`, lo que obligaba a recompilar para apuntar a otro servidor y hacía que la contraseña viajara dentro del artefacto entregado al cliente. Ahora **cambiar de servidor de base de datos no requiere recompilar**: se edita el archivo `.properties`, o se borra para que la aplicación vuelva a pedir los datos en el diálogo de arranque.

SEGURIDAD

`ConfiguracionBD.guardar()` escribe el archivo con permisos `rw-----` en sistemas POSIX. La contraseña queda en claro; en Windows hay que restringir el ACL a mano.

Qué configura entonces `persistence.xml`

`src/main/resources/META-INF/persistence.xml` define la unidad de persistencia `pinkyPuffPersistence` (`RESOURCE_LOCAL`), lista las cinco entidades y ajusta Hibernate — todo menos las credenciales:

```
hibernate.hbm2ddl.auto           = update    Crea/actualiza tablas al arrancar
hibernate.connection.provider_class = HikariCPConnectionProvider
hibernate.hikari.minimumIdle     = 2
hibernate.hikari.maximumPoolSize = 10
hibernate.hikari.idleTimeout     = 30000    ms
hibernate.hikari.connectionTimeout = 10000  ms
hibernate.jdbc.batch_size        = 30        Agrupa INSERT/UPDATE
hibernate.order_inserts/updates  = true      Necesario para el lote
hibernate.jdbc.fetch_size        = 100
hibernate.show_sql / format_sql  = false     ← ponga true para depurar
```

La inyección de las credenciales ocurre en `persistencia/JPAUtil.java`:

```
Persistence.createEntityManagerFactory(
    "pinkyPuffPersistence",
    ConfiguracionBD.cargar().propiedadesJpa()); // ← host/usuario/clave
```

8. Paso 5 — Compilar con Maven

```
mvn clean compile           # compila a target/classes
mvn clean package           # genera target/PinkyPuff.jar (fat JAR, ~39 MB)
mvn clean package -Pdist-windows # además genera PinkyPuff.exe
```

Qué hace cada plugin

maven-compiler-plugin — compila con `maven.compiler.release = 21` y registra Lombok en `annotationProcessorPaths`. Sin ese registro, Lombok no genera los getters y la compilación falla con cientos de errores "cannot find symbol".

maven-shade-plugin — produce el JAR único con todas las dependencias dentro. Se eligió `*shade*` y no `*assembly*` por una razón concreta, anotada en el propio `pom.xml`: Hibernate se apoya en archivos `META-INF/services/*` repartidos entre muchos JAR y hay que **fusionarlos**. Assembly deja que el último sobrescriba a los anteriores y el arranque revienta. `ServicesResourceTransformer` hace esa fusión.

Los filtros del shade excluyen:

- `META-INF/*.SF, *.DSA, *.RSA` — las firmas digitales de los JAR originales dejan de ser válidas al refundirlos; si se quedan, Java rechaza el JAR entero.
- `module-info.class` — el JAR se ejecuta por classpath, no por módulos JPMS.

launch4j-maven-plugin (solo en el perfil `dist-windows`) — envuelve el JAR en un `.exe` con icono, versión y comprobación de que hay Java 21; si falta, abre la página de descarga de Adoptium. El plugin trae sus propios binarios, así que el `.exe` puede generarse desde Linux.

Perfiles de JavaFX

JavaFX es la única dependencia con binarios nativos por sistema operativo. Los perfiles `javafx-linux`, `javafx-windows` y `javafx-mac` **se activan solos** según el SO y fijan la propiedad `javafx.platform`, que se usa como classifier. No hay que hacer nada manualmente.

El perfil `dist-windows` añade además los artefactos JavaFX con clasificador `win` **junto** a los de Linux: las clases Java son idénticas y los nativos tienen nombres distintos, así que conviven dentro del mismo JAR y el resultado funciona en ambos sistemas.

9. Paso 6 — Crear el primer administrador

Sin un administrador no se puede pasar de la pantalla de login. Hay dos caminos.

Opción de desarrollo

Ejecutar `utils/CrearAdministradorPrueba.java` como clase principal. Crea:

```
email:    admin@correo.es
usuario:  admin
clave:    contraxd
```

DOCUMENTACIÓN DESVIADA

El README.md menciona un archivo `utils/CrearUsuarioPrueba.java` que no existe. El nombre correcto es `CrearAdministradorPrueba`.

Opción de producción

SQL directo, con el hash calculado por PostgreSQL:

```
INSERT INTO administrador (email, username, password_hash)
VALUES ('admin@suempresa.com', 'admin',
       encode(sha256('SU_CONTRASEÑA'::bytea), 'hex'));
```

NOTA DE SEGURIDAD

El sistema aplica un solo pase de SHA-256 sin sal ni estiramiento, lo que lo hace vulnerable a tablas arcoíris y a fuerza bruta con GPU. pgcrypto ya está instalado por instalar_bd.sql y ofrece crypt() con blowfish; migrar a bcrypt o Argon2 exigiría convertir los hashes existentes.

10. Paso 7 — Ejecutar y verificar

```
mvn compile exec:java          # ejecuta usando exec.mainClass = view.FrmLogin
java -jar target/PinkyPuff.jar # ejecuta el artefacto empaquetado
```

Qué debe ocurrir, en orden

1. EstiloUI.aplicarFlatLaf()
Instala el Look and Feel y la paleta de marca.
2. DialogoConexion.asegurarConexion()
Carga ~/.pinky puff/conexion.properties → prueba la conexión con DriverManager → si falla (o si el archivo no existe) abre el diálogo modal para corregir host/puerto/base/usuario/clave, con botón "Probar conexión". Al guardar llama a JPAUtil.reiniciar() y reintenta. La aplicación NUNCA llega a Hibernate sin una conexión verificada.
3. Aparece FrmLogin.
4. El usuario se autentica → SELECT ... FROM fn_autenticar_admin(...)
Una sola ida a la base: verifica la contraseña, registra el intento en sesiones_admin, aplica el bloqueo (3 intentos fallidos / ventana de 15 min) y devuelve el mensaje ya redactado.
5. FrmDashboard: CardLayout con 6 tarjetas (pedidos, todos, clientes, reporte, config, credenciales) y la barra de navegación JavaFX.
6. ControladorDashboard.inicializar()
repoEtiqueta.inicializarPorDefecto() ← siembra TIPO y PRECIO_RAPIDO si la tabla etiquetas_config está vacía.

Que la aplicación llegue al dashboard y muestre las etiquetas por defecto es la señal de que **toda la cadena** —Docker, PostgreSQL, esquema, credenciales, Hibernate e interfaz— está en pie.

11. Verificación de la infraestructura

#	Comprobación	Comando o acción	Resultado esperado
1	JDK correcto	java -version	21.x
2	Maven disponible	mvn -version	3.8+ y JDK 21
3	Contenedor en marcha	docker ps	pinky puff-db en estado Up
4	Puerto expuesto	pg_isready -h localhost -p 5433	accepting connections
5	Tablas creadas	\dt en psql	7 tablas base + tablas de apoyo

#	Comprobación	Comando o acción	Resultado esperado
6	Funciones creadas	\df fn_* en psql	fn_autenticar_admin, fn_evolucion_mensual...
7	Extensiones	\dx en psql	pg_trgm, pgcrypto, pg_stat_statements
8	Compilación	mvn clean package	target/PinkyPuff.jar (~39 MB)
9	Configuración	~/pinkyPuff/conexion.properties	existe y con puerto 5433
10	Administrador	SELECT * FROM administrador;	al menos una fila
11	Arranque	java -jar target/PinkyPuff.jar	login → dashboard con 6 pestañas
12	Reporte PDF	abrir la pestaña de reporte	se genera sin error de función inexistente

Consultas útiles de verificación

```
\dt -- tablas del esquema público
\d clientes -- estructura de una tabla
SELECT COUNT(*) FROM clientes;
SELECT * FROM etiquetas_config WHERE categoria = 'TIPO' ORDER BY orden;

SELECT p.id, c.nombre, p.estado, p.fecha_registro
FROM pedidos p
JOIN clientes c ON p.cliente_id = c.id
ORDER BY p.fecha_registro DESC;
```

Alternativa gráfica: pgAdmin

6. Instalar pgAdmin desde su sitio oficial.
7. Clic derecho en "Servers" → "Register" → "Server".
8. Pestaña **General**: nombre libre (por ejemplo "PinkyPuff Local").
9. Pestaña **Connection**: Host [localhost](#), Port [5433](#), Maintenance database [postgres](#), Username [admin](#), Password [contra](#).
10. Guardar. Las tablas están en Servers → PinkyPuff Local → Databases → postgres → Schemas → public → Tables.

12. Empaquetado y entrega al cliente

```
mvn clean package -Pdist-windows
cp target/PinkyPuff.jar target/PinkyPuff.exe dist/
```

Entregue la carpeta [dist/](#) completa:

Archivo	Para qué
PinkyPuff.jar	Ejecutable multiplataforma
PinkyPuff.exe	Ejecutable de Windows con verificación de Java 21
PinkyPuff.sh / PinkyPuff.bat	Lanzadores
instalar_bd.sql	Script idempotente para crear o actualizar la base

Archivo	Para qué
conexion.properties.ejemplo	Plantilla que el instalador copia a ~/.pinkypuff/
INSTALACION.md	Manual del instalador

CONSIDERACIÓN PARA PRODUCCIÓN

hibernate.hbm2ddl.auto = update es cómodo en desarrollo pero arriesgado al entregar: nunca borra ni renombra columnas, y puede desviarse silenciosamente del esquema esperado. instalar_bd.sql es el esquema autoritativo; para distribución conviene evaluar el cambio a validate.

13. Operación y mantenimiento

Ciclo de vida del contenedor

```
docker compose up -d      # levantar
docker compose down       # parar (los datos se conservan en el volumen)
docker compose down -v    # parar Y BORRAR los datos
```

Mantenimiento de la base

```
CALL sp_mantenimiento_bd();    -- semanalmente o al cierre del día
CALL sp_refrescar_dashboard(); -- refrescar vistas materializadas a demanda
```

`sp_mantenimiento_bd()` actualiza estadísticas del planificador, refresca las vistas materializadas y purga bitácoras antiguas.

Actualizar el esquema

Como `instalar_bd.sql` es idempotente, para aplicar cambios basta con volver a ejecutarlo:

```
docker exec -i pinkypuff-db psql -U admin -d postgres < dist/instalar_bd.sql
```

REGLA

Si se toca la base de datos desde el código, hay que actualizar `dist/instalar_bd.sql` en el mismo cambio. Es el único esquema autoritativo.

Cambiar de servidor de base de datos sin recompilar

Edite `~/.pinkypuff/conexion.properties`, o borre el archivo para que la aplicación vuelva a pedir los datos en el diálogo de arranque.

Desbloquear una cuenta

El bloqueo se activa a los 3 intentos fallidos y caduca solo a los 15 minutos. Para desbloquear de inmediato:

```
DELETE FROM sesiones_admin WHERE usuario_ingresado = 'admin' AND exito = FALSE;
```

14. Resolución de problemas

Síntoma	Causa probable	Solución
function fn_autenticar_admin(...) does not exist	No se ejecutó dist/installar_bd.sql. Hibernate creó las tablas pero no las funciones	Ejecutar el script (sección 6)
function fn_evolucion_mensual(...) does not exist al abrir el reporte	Misma causa	Ídem
Cientos de errores "cannot find symbol" al compilar	Lombok no se registró como procesador de anotaciones	Verificar annotationProcessorPaths en pom.xml; usar JDK 21 y mvn clean
El JAR se genera pero no arranca (error de servicios de Hibernate)	Se usó assembly en vez de shade, o se perdió ServicesResourceTransformer	Recompilar con mvn clean package sin alterar el shade-plugin
Java rechaza el JAR ("invalid signature")	Quedaron firmas META-INF/*.SF, *.DSA o *.RSA dentro del fat JAR	Revisar los filtros del shade-plugin
El diálogo de conexión aparece siempre	El .properties no se guarda, o apunta a un host o puerto erróneo	Confirmar puerto 5433, contenedor Up y permisos de escritura en ~/.pinkypuff/
Connection refused al probar la conexión	Contenedor caído o puerto equivocado (5432 en vez de 5433)	docker ps; pg_isready -h localhost -p 5433
Conflicto de puerto al levantar Docker	Hay otro servicio ocupando el 5433	Cambiar el mapeo en docker-compose.yml y en conexion.properties
permission denied to create extension	El usuario no es superusuario en el servidor destino	Que un administrador cree antes pg_trgm y pgcrypto
Cuenta bloqueada tras 3 intentos	Bloqueo aplicado por fn_autenticar_admin	Esperar 15 minutos o ejecutar el DELETE de la sección 13
El .exe avisa de que falta Java	La máquina no tiene Java 21	Instalar el JDK 21 (el propio .exe abre Adoptium)
Se perdieron los datos	Se ejecutó docker compose down -v	El volumen se borró: reinstalar el esquema y recrear el administrador

Depuración de consultas

Para ver el SQL que genera Hibernate, en `META-INF/persistence.xml`:

```
hibernate.show_sql = true
hibernate.format_sql = true
```

15. Referencia rápida de comandos

Base de datos

```
docker compose up -d          # levantar PostgreSQL
docker ps                    # estado del contenedor
pg_isready -h localhost -p 5433 # ¿responde?
docker exec -it pinkypuff-db psql -U admin -d postgres # consola SQL
docker exec -i pinkypuff-db psql -U admin -d postgres < dist/installar_bd.sql
docker compose down          # parar
docker compose down -v       # parar y BORRAR datos
```

Compilar y ejecutar

```
mvn clean compile
mvn clean package # target/PinkyPuff.jar
mvn clean package -Pdist-windows # además PinkyPuff.exe
mvn compile exec:java # ejecuta view.FrmLogin
java -jar target/PinkyPuff.jar
java -Dpinkypuff.config=/otra/ruta.properties -jar target/PinkyPuff.jar
```

Mantenimiento de la base

```
CALL sp_mantenimiento_bd();
CALL sp_refrescar_dashboard();
DELETE FROM sesiones_admin WHERE usuario_ingresado='admin' AND exito=FALSE;
```

Rutas clave de infraestructura

Qué busco	Dónde está
Punto de entrada	view/FrmLogin.java → main()
Credenciales de la base	~/.pinkypuff/conexion.properties
Lectura de esas credenciales	utils/ConfiguracionBD.java
Configuración de Hibernate	src/main/resources/META-INF/persistence.xml
Fábrica de EntityManager	persistencia/JPAUtil.java
Diálogo de conexión de arranque	view/DialogoConexion.java
Esquema completo de la base	dist/instalar_bd.sql
Build y perfiles de distribución	pom.xml

16. Advertencias sobre archivos engañosos

Durante la instalación es fácil editar el archivo equivocado. Estos son los que **no** hay que tocar:

Archivo	Estado real
src/Other Sources/resources/META-INF/persistence.xml	Vacío y con el nombre mal escrito ("persistance"). Hibernate no lo lee. El bueno es src/main/resources/META-INF/persistence.xml. Candidato a borrar.
src/main/java/utils/configuraciones.xml	Obsoleto y engañoso. Apunta a una base "system" que no existe, con la contraseña en claro, y ningún código lo lee. Candidato a borrar.
schema.sql (raíz)	Versión antigua y parcial del esquema. Sirve para leer, no para instalar. Use dist/instalar_bd.sql.
README.md, sección "Parámetros de conexión"	Desactualizado: dice que las credenciales están en persistence.xml, lo cual ya no es cierto. También cita CrearUsuarioPrueba, que no existe (es CrearAdministradorPrueba).

Documento derivado de MANUAL_TECNICO.md y DOCUMENTACION_TECNICA.md del proyecto PinkyPuff. Todo el contenido procede de esas fuentes; no se ha añadido información no verificada en ellas.